PROJECT DOCUMENTATION

Metro Ticket Generating System

ServiceNow Application Project

Document Date	16 October 2026	**Version**	1.0

Document purpose

This document describes the purpose, proposed architecture, setup approach, functional flow, testing, and future improvements for the Metro Ticket Generating System in ServiceNow. It follows the section order of the supplied Project Documentation Format while replacing MERN-specific instructions with ServiceNow-relevant content.

1. Introduction

Project Title: Metro Ticket Generating System in ServiceNow.

Team Members and Roles: Use the actual names assigned by the project team. Suggested responsibilities are ServiceNow Developer, Application Administrator, Tester, and Project Coordinator. Names are not supplied in the source material, so they are not invented here.

2. Project Overview

Purpose: Provide a structured way to submit metro ticket requests, validate required details, create a ticket record with a reference, track its status, and let authorized staff review and manage records within ServiceNow.

Key Features:

• Ticket request form for entering journey/ticket information.
• Required-field and input validation.
• Creation and storage of a ticket record in ServiceNow.
• Ticket reference and submission confirmation.
• Status tracking using configured workflow logic.
• Role-based access and authorized staff review.

Assumption: Exact fields, metro fare calculation, payment, QR scanning, and external metro integrations are included only if separately implemented and approved. They are not confirmed by the available project requirements.

3. Architecture

The application is designed around the ServiceNow platform rather than a separate React/Node.js/MongoDB stack.

Layer	Proposed ServiceNow Component	Responsibility
User interface	ServiceNow form / configured workspace or portal	Collect ticket details and display record/confirmation information.
Application logic	Business Rules, Flow Designer, client-side validation where needed	Validate submissions, apply workflow logic, and manage ticket status.
Data storage	ServiceNow table and fields for ticket records	Store ticket details, unique record number, status, and audit information.
Security	Roles, groups, and Access Control Rules (ACLs)	Restrict viewing and updating records to authorized users.
Notifications / integrations	Only if configured for this project	Email or external connections are optional and should not be assumed.

Typical workflow

User opens the ticket form → enters required details → system validates input → ServiceNow creates a ticket record → user receives a reference/confirmation → authorized staff review or update status.

4. Setup Instructions

Prerequisites: Access to an authorized ServiceNow development or test instance; appropriate developer/admin permissions; the application scope and ticket table/form configured; and test accounts for user and staff roles.

Setup steps:

1. Sign in to the approved ServiceNow instance.
2. Open the relevant application scope or create one according to the project setup.
3. Configure the ticket table and fields based on the approved requirements.
4. Build the ticket form and add required-field validation.
5. Configure record creation, reference visibility, and status workflow using the selected ServiceNow tools.
6. Configure roles and ACLs for user and staff access.
7. Test the workflow in the development/test instance before demonstrating it.

Do not place passwords, secret tokens, or private instance credentials in this document. Instance URL and access details should be filled in by the project administrator.

5. Folder Structure / Application Organization

Unlike a MERN application, a ServiceNow project is usually organized through platform application artifacts rather than local client/server folders. Use the following logical organization as a documentation guide; exact artifact names depend on what has been created in the instance.

Artifact group	Example contents	Purpose
Data model	Ticket table, fields, reference fields	Defines the structure of ticket records.
User interface	Ticket form, list view, workspace/portal page if used	Supports entry and review of ticket details.
Business logic	Business Rules, Flow Designer flows, validation scripts if used	Implements validation and status transitions.
Security	Roles, groups, ACL rules	Controls authorized access.
Testing and delivery	Test cases, defect log, update set/application package if used	Supports verification and controlled movement between instances.

6. Running the Application

1. Open the project's configured ServiceNow instance using an authorized account.
2. Navigate to the Metro Ticket Generating System application/module or configured entry point.
3. Open the ticket request form, enter valid details, and submit.
4. Confirm that the record is created and a ticket reference is shown.
5. Use an authorized staff account to verify that the record can be reviewed and its status managed.

There are no standard `npm start` commands for this platform-based implementation. The exact navigation path depends on the application's configured module, portal, or workspace.

7. API Documentation

No custom API endpoints were specified in the available project requirements. If the implementation exposes an endpoint, document it using the table below. Do not list a route as implemented until it has been created and verified.

Item	Documented value
Endpoint / resource	To be completed only if a custom or Table API endpoint is used.
HTTP method	GET / POST / PATCH / DELETE, as actually configured.
Authentication / access	ServiceNow authentication and permissions applicable to the instance and integration.
Request / response example	Add a sanitized example from the implemented endpoint; exclude credentials and personal data.

8. Authentication and Authorization

Access should be controlled using ServiceNow's configured authentication and authorization mechanisms. Assign only the roles required for each responsibility and use ACLs to protect ticket records and fields. Validate that ordinary users can perform only permitted actions and that staff/admin users can review or update records according to their assigned permissions. The specific login method and role names must match the deployed instance.

9. User Interface

The user interface should make the ticket submission flow clear and consistent. At minimum, document the ticket form, validation messages, successful submission confirmation, ticket reference, ticket list/record view for authorized staff, and status display if implemented.

Screenshots to add from the actual application:

- Ticket request form.
- Example of required-field validation.
- Successful ticket submission and reference.
- Ticket record/status view for authorized staff.

Screenshots are not included because no screenshots of the running application were supplied. Add genuine project screenshots before final submission.

10. Testing

Test the application in an authorized development/test instance before accepting it. Record actual outcomes in the UAT test log; do not mark a test as passed until it has been executed.

Test ID	Scenario	Expected result
TC-001	Open ticket form	Form is accessible to an authorized user.
TC-002	Submit valid ticket details	Ticket record is created successfully.
TC-003	Leave a required field empty	Validation identifies missing information and blocks invalid submission.
TC-004	Review ticket reference/confirmation	User can identify the created request using the configured reference.
TC-005	Review status handling	Status is saved and displayed according to configured workflow.
TC-006	Check access with different roles	Permissions match the configured role and ACL rules.

Suggested approach: functional testing, positive and negative validation tests, role/permission checks, and UAT with representative users. Testing tools and automated test suites should be listed only if actually used.

11. Screenshots or Demo

Demo sequence: (1) sign in, (2) open the ticket form, (3) enter and submit ticket details, (4) show the generated record/reference, and (5) demonstrate authorized staff review/status handling. Add the actual demo URL or recording location only if one exists and is approved for sharing.

Demo link / location: To be added by the project team.

12. Known Issues

No verified defect list or completed test results were provided while preparing this document. Therefore, no specific known bug is asserted here. Record confirmed issues, reproduction steps, severity, workaround, and status after testing.

Potential configuration checks (not confirmed defects): verify required-field rules, unique ticket reference behavior, role permissions, and status transitions in the deployed instance.

13. Future Enhancements

Possible enhancements, subject to project approval and platform capability, include:

- QR code on a ticket for faster verification.
- Payment integration, if online payment is in scope.
- Metro timetable or fare integration through an approved external service.
- Email or in-platform notifications for ticket confirmation and status changes.
- Reporting dashboards for ticket volumes and statuses.
- Improved accessibility and multilingual user-interface support.

Implementation note: This document is adapted from the supplied MERN-oriented documentation template. ServiceNow-specific architecture and steps are used instead of React, Node.js, Express.js, MongoDB, and npm commands. Any feature or integration not confirmed in the project requirements is marked as optional or pending.